

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

Кафедра «Вычислительные системы и технологии»

ОТЧЕТ

по лабораторной работе №2

по дисциплине

Языки интернет-программирования

РУКОВОДИТЕЛЬ:

(подпись)

Яшнов М.Д.
(фамилия, и.,о.)

СТУДЕНТ:

(подпись)

Рыжов А.С.
(фамилия, и.,о.)

23-ИИ
(шифр группы)

Работа защищена «__» _____

С оценкой _____

Нижний Новгород
2026

Ссылка на GitHub: <https://github.com/googlehater/barbershop>

Тема: Реализация проекта с серверным рендерингом страниц, работа с базой данных и безопасностью.

Цель:

- Освоить серверный рендеринг HTML-страниц
- Научиться безопасно работать с базой данных через чистый SQL
- Использовать архитектуру MVC и принципы DI
- Освоить контейнеризацию с Docker для приложения и БД
- Реализовать CRUD операции и взаимосвязь таблиц

Лабораторная работа выполнялась по порядку заданий:

1. Серверный рендеринг HTML

Задание: все HTML-страницы должны формироваться на сервере. Клиентский JavaScript разрешён только для интерактивности

Ход работ:

Было решено реализовать server-side rendering при помощи Flask и Jinja

Реализовано это следующим образом (на примере страницы message.html – страницы записи клиента)

Src/controllers/appointment_controller.py:

```
'''
```

```
@app.route("/appointment", methods=["GET", "POST"])
```

```
def appointment():
```

```
    print("appointment route called")
```

```
...
```

```
    services = service_model.get_all_services()
```

```
    return render_template("message.html", services=services)
```

```
'''
```

Тут как раз и происходит рендеринг. Сценарий такой: пользователь открывает /appointment, Flask вызывает appointment(), а уже из базы данных подгружаются услуги:

```
''' python

services = service_model.get_all_services()

'''
```

(об этом позже в отчете).

После этого Jinja подставляет значения в HTML следующим образом:

```
<div class="form-row">
  <div class="form-group">
    <label for="service">Услуга *</label>
    <select id="service" name="service" required>
      <option value="" disabled selected>Выберите</option>
      {% for service in services %}
      <option value="{{ service.id }}">{{ service.services_name }} -- {{ service.price }}</option>
      {% endfor %}
    </select>
  </div>
</div>
```

0.1: Цикл Jinja, который выполняется на сервере. Цикл выделен красной рамкой

Подобным образом реализованы и другие контроллеры:

```
src > controller > home_controller.py > ...
1  from flask import render_template
2  from app import app
3
4
5  @app.route("/")
6  def index():
7      return render_template("index.html")
8
```

1: рендеринг главной страницы

```
src > controller > about_us_controller.py > ...
1  from flask import render_template
2  from app import app
3
4
5  @app.route("/about_us")
6  def about_us():
7      return render_template("about_us.html")
8
```

2: рендеринг страницы "о нас"

ЗАПИСЬ

Заполните форму — мы подтвердим запись

Имя *

Телефон *

Услуга *

Выберите

Выберите

Мужская стрижка -- 1500.00

Стрижка бороды -- 1000.00

Стрижка + борода -- 2200.00

Бритьё опасной бритвой -- 1200.00

Стрижка ножницами -- 1800.00

Дата *

Мастер

Любой

ЗАПИСАТЬСЯ

3: что по итогу получает браузер

Важно: браузер не выполняет цикл — он получает полностью сформированную страницу

```
<div class="form-row">
  <div class="form-group">
    <label for="service">Услуга *</label>
    <select id="service" name="service" required>
      <option value="" disabled selected>Выберите</option>

      <option value="1">Мужская стрижка -- 1500.00</option>
      <option value="2">Стрижка бороды -- 1000.00</option>
      <option value="3">Стрижка + борода -- 2200.00</option>
      <option value="4">Бритьё опасной бритвой -- 1200.00</option>
      <option value="5">Стрижка ножницами -- 1800.00</option>

    </select>
  </div>
  <div class="form-group">
    <input type="text" value="дд.мм.гггг">
  </div>
  <div class="form-group">
    <div>
      <div>Любой</div>
      <div></div>
    </div>
  </div>
</div>
```

4: исходный код страницы

Как видно по исходному коду страницы (ctrl+u), она уже содержит реальный `<option>` без шаблонных конструкций.

2. Развертывание БД через Docker Compose

Задание: развернуть БД и приложения через Docker Compose

Ход работ:

Для выполнения этого задания был создан файл `docker_compose.yaml`:

```

👉 docker-compose.yml
1  services:
2    db:
3      image: postgres:17-alpine
4      environment:
5        POSTGRES_USER: ${DB_USER}
6        POSTGRES_PASSWORD: ${DB_PASSWORD}
7        POSTGRES_DB: ${DB_NAME}
8      ports:
9        - "${DB_PORT}:5432"
10     volumes:
11       - pgdata:/var/lib/postgresql/data
12       - ./src/database:/docker-entrypoint-initdb.d
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
15       interval: 5s
16       timeout: 5s
17       retries: 5

```

```

19     backend:
20       build:
21         context: .
22         dockerfile: Dockerfile
23       environment:
24         DB_HOST: db
25         DB_PORT: 5432
26         DB_USER: ${DB_USER}
27         DB_PASSWORD: ${DB_PASSWORD}
28         DB_NAME: ${DB_NAME}
29         FLASK_ENV: development
30         FLASKDEBUG: 1
31       ports:
32         - "8080:80"
33       volumes:
34         - ./src:/app # all folder prjct
35         - /app/__pycache__ # cache ignore
36       depends_on:
37         db:
38           condition: service_healthy
39       restart: on-failure
40       working_dir: /app
41
42     volumes:
43       pgdata:
44

```

Веб-приложение Flask работает внутри контейнера и прослушивает порт 5000. В файле docker-compose.yml этот порт опубликован как 5000:5000, что обеспечивает доступ к приложению по адресу <http://localhost:5000>. Сервер

PostgreSQL работает в отдельном контейнере и использует стандартный порт 5432. Приложение подключается к базе данных по имени сервиса db и порту 5432 через внутреннюю сеть Docker Compose.

```
Dockerfile
1 FROM python:3.13-alpine
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY src/ .
9 EXPOSE 80
10
11 CMD ["python", "app.py"]
```

3. MVC архитектура

Задание: использование MVC архитектуры, контроллеры обрабатывают запросы, модели — работу с БД, представления — шаблоны HTML

Ход работы:

MVC – архитектурный шаблон model-view-controller разделяет приложение на 3 независимые друг от друга части:

- модель работа с базой данных и SQL-запросами
- view (представления) HTML шаблоны для отображения данных
- controller отвечает за обработку http запросов и координацию работы приложения

Архитектура (для создания использовалось расширение для VS Code «Project Tree»):

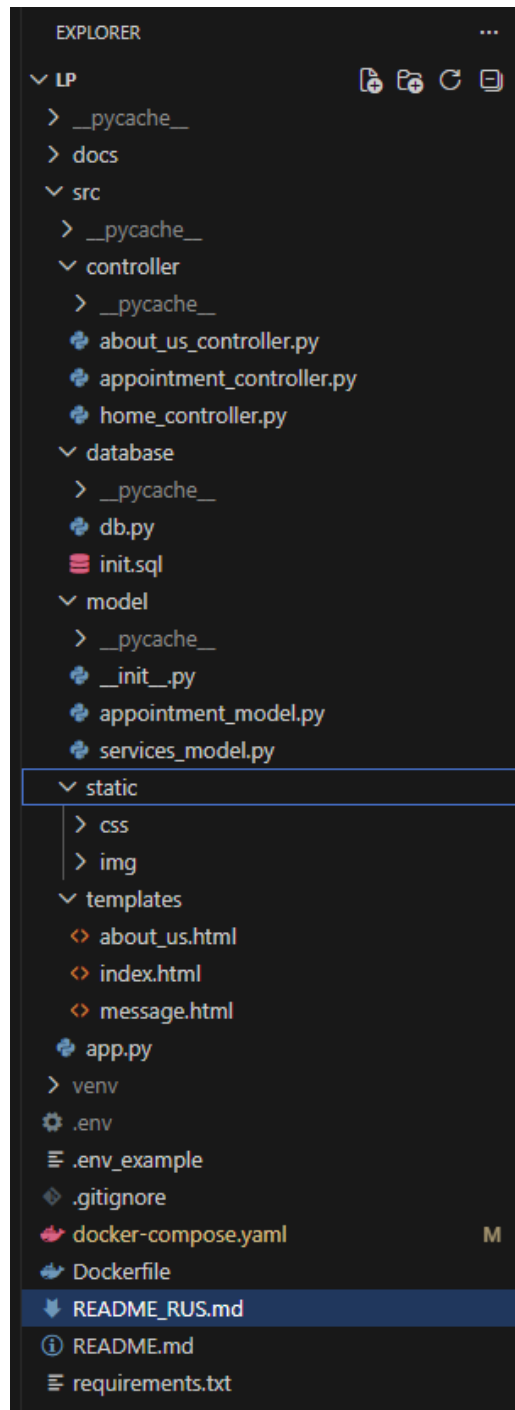
Fade

```
├─ docker-compose.yaml
├─ Dockerfile
├─ docs
│   └─ Otchet_Ryzhov_23-II_1.pdf
│   └─ Задание к лабе 1.txt
│   └─ Задание к лабе 2.txt
```

- ├─ README.md
- ├─ README_RUS.md
- ├─ requirements.txt
- └─ src
 - ├─ app.py
 - ├─ controller
 - | └─ about_us_controller.py
 - | └─ appointment_controller.py
 - | └─ home_controller.py
 - ├─ database
 - | └─ db.py
 - | └─ init.sql
 - ├─ model
 - | └─ appointment_model.py
 - | └─ services_model.py
 - | └─ __init__.py
 - ├─ static
 - | └─ css
 - | └─ style.css
 - | └─ img
 - | └─ barbershop_Interior.jpg
 - | └─ barbershop_vibe.jpg
 - | └─ Beard_care.jpg
 - | └─ haircut_1.jpg
 - | └─ haircut_2.jpg
 - | └─ haircut_3.jpg
 - | └─ like_a_model_cut.jpeg
 - | └─ Shaving_with_a_straight_razor.png
 - | └─ tape_1.jpg

- └─ templates
 - └─ about_us.html
 - └─ index.html
 - └─ message.html

Или же скриншот:



5 : MVC архитектура проекта


```

from flask import render_template, request, redirect, url_for
from app import app, connection
from model.services_model import service_model

from model.services_model import service_model
from model.appointment_model import AppointmentModel

appointment_model = AppointmentModel(connection)

@app.route("/appointment", methods=["GET", "POST"])
def appointment():
    print("appointment route called")
    print(f'request method: {request.method}')

    if request.method == "POST":
        print(f"POST data: {request.form}")
        name = request.form.get("name", "").strip() # удаляем лишние пробелы
        phone = request.form.get("phone", "").strip()
        service_id = request.form.get("service", "").strip()
        date = request.form.get("date")
        time = request.form.get("time")
        comment = request.form.get("comment", "").strip()
        master_id = request.form.get("master") or 1

        if not name:
            return "Name is required", 400

        if len(phone) < 10:
            return "phone is too short", 400

```

6: контроллер записи на стрижку

```

c:\model > appointment_model.py > AppointmentModel > create_appointment
18 class AppointmentModel:
21     def _execute(self, query, params=None, fetch=True, fetch_one=False, commit=True):
40         conn.rollback()
41         raise e
42
43     def get_all_appointments(self):
44         query = "SELECT * FROM appointments ORDER BY id"
45         return self._execute(query)
46
47     def get_by_id(self, id):
48         query = "SELECT * FROM appointments WHERE id = %s"
49         return self._execute(query, (id,), fetch_one=True)
50
51     def get_or_create_client(self, client_name, phone):
52         """
53         looks for client by phone
54         if not found -> creates new one
55         returns client_id
56         """
57
58         query_select = """
59             SELECT id
60             FROM clients
61             WHERE phone = %s
62         """
63
64         result = self._execute(query_select, (phone,), fetch_one=True)
65
66         if result:
67             return result['id']
68
69         # Создание нового клиента
70         query_insert = """
71             INSERT INTO clients (client_name, phone)
72             VALUES (%s, %s)
73             RETURNING id
74         """
75
76         result = self._execute(query_insert, (client_name, phone), fetch_one=True, commit=True)
77
78         if result:
79             return result['id']
80         else:
81             raise Exception("failed to create client")

```

7: модель с "голым" SQL

4. CRUD

Задание: CRUD операции минимум для одной сущности

Ход работ:

Для сущности «Услуга» реализован полный набор CRUD-операций. Создание записи выполняется SQL-запросом INSERT, чтение данных — запросом SELECT, обновление — запросом UPDATE, удаление — запросом DELETE. Все операции реализованы в модели `services_model.py`, вызываются из контроллера и доступны через веб-интерфейс приложения.

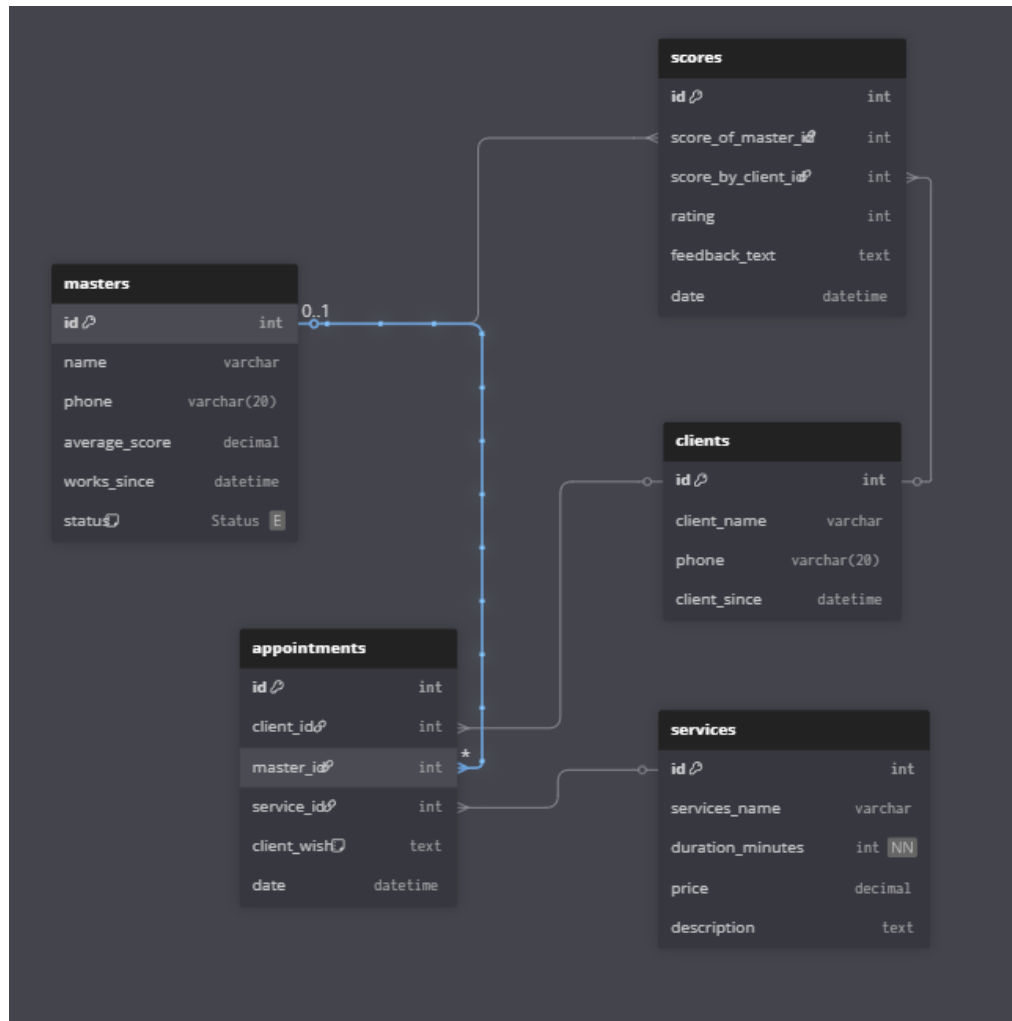
```
5 class ServiceModel:
36
37     # crud
38
39     def get_all_services(self):
40         query = "SELECT * FROM services ORDER BY id"
41         return self._execute(query)
42
43     def get_by_id(self, id):
44         query = "SELECT * FROM services WHERE id = %s"
45         return self._execute(query, (id,), fetch_one=True)
46
47     def create_service(self, name, duration, price, description=None):
48         query = """
49         INSERT INTO services (services_name, duration_minutes, price, description)
50         VALUES (%s, %s, %s, %s) RETURNING id
51         """
52         result = self._execute(query, (name, duration, price, description), fetch_one=True)
53         return result['id'] if result else None
54
55     def update_service(self, service_id, name=None, duration=None, price=None, description=None):
56         fields = {}
57         if name is not None:
58             fields['services_name'] = name
59         if duration is not None:
60             fields['duration_minutes'] = duration
61         if price is not None:
62             fields['price'] = price
63         if description is not None:
64             fields['description'] = description
65
66         if not fields:
67             return False
68
69         set_clause = ", ".join([f"{k} = %s" for k in fields.keys()])
70         values = list(fields.values()) + [service_id]
71         query = f"UPDATE services SET {set_clause} WHERE id = %s"
72         self._execute(query, values, fetch=False)
73
74         return True
75
76     def delete_service(self, service_id):
77         query = "DELETE FROM services WHERE id = %s"
78         self._execute(query, (service_id,), fetch=False)
79
80 from database.db import connect_db
81 service_model = ServiceModel(connect_db) # тут DI, модель не знает откуда connection!
82
```

5. Три таблицы с нормализацией и связями

Задание: минимум три таблицы по предметной области. Не менее одной связи между таблицами (один-ко-многим или многие-ко-многим). Если нет идеи для предметной области — сделать таблицы для статистики действий/логов/переходов. Все таблицы должны быть в нормальных формах

Ход работы:

Первым делом было решено воспользоваться сервисом dbml.dbdiagram.io, потому что в нем достаточно легко проектировать базы данных.



9: схема БД

Позже псевдо язык этого сервиса был переведен в обычный SQL, и записан в `src/database/init.sql`, чтобы поднятию контейнера в докере создавалась база данных, при условии, что она уже не создана.

```
src > database > init.sql
1  -- 1. create enum for master status type
2  CREATE TYPE master_status AS ENUM ('fired', 'works', 'on_vacation')
3
4  -- 2. table services
5  CREATE TABLE services (
6      id SERIAL PRIMARY KEY,
7      services_name VARCHAR(100) NOT NULL,
8      duration_minutes INT NOT NULL,
9      price DECIMAL(10,2) NOT NULL,
10     description TEXT
11 );
12
13 -- 3. table masters
14 CREATE TABLE masters (
15     id SERIAL PRIMARY KEY,
16     name VARCHAR(100) NOT NULL,
17     phone VARCHAR(20),
18     average_score DECIMAL(3,2), -- средний рейтинг от 1 до 5
19     works_since DATE,
20     status master_status DEFAULT 'works'
21 );
22
23 -- 4. table clients
24 CREATE TABLE clients (
25     id SERIAL PRIMARY KEY,
26     client_name VARCHAR(100) NOT NULL,
27     phone VARCHAR(20) NOT NULL,
28     client_since DATE DEFAULT CURRENT_DATE
29 );
30
31 -- 5. table appointments for services
32 CREATE TABLE appointments (
33     id SERIAL PRIMARY KEY,
34     client_id INT NOT NULL,
```

10: чисты SQL в init.sql

6. Чистый SQL

Задание: все запросы выполняются только через чистый SQL, без ORM

Ход работы:

Все запросы выполняются только через чистый SQL, без ORM. Это уже показано на скриншоте 8 и не только. Для удобства использования была добавлена закрытый метод класса `_execute()`, чтобы выполнять SQL

7. Защита от SQL-инъекций

Задание: защита от SQL-инъекций обязательна (использование параметризованных запросов)

Ход работы:

Для защиты от SQL-инъекций все SQL-запросы в проекте выполняются с использованием параметризованных запросов. В SQL-коде используются плейсхолдеры %s, а пользовательские данные передаются отдельным набором параметров в метод `_execute()`. Это исключает возможность внедрения SQL-кода через ввод пользователя. Дополнительно в контроллерах выполняется проверка входных данных, например, идентификаторы записей проверяются как числовые значения (видно на скриншоте выше №5 или ниже).

```
5 class ServiceModel:
47 def create_service(self, name, duration, price, description=None):
48     query = """
49     INSERT INTO services (services_name, duration_minutes, price, description)
50     VALUES (%s, %s, %s, %s) RETURNING id
51     """
52     result = self._execute(query, (name, duration, price, description), fetch_one=True)
53     return result['id'] if result else None
54
```

11: защита от SQL инъекций при помощи %s и методом _execute()

8. Защита от XSS

Задание: обязательная защита от XSS

Ход работы:

XSS – это Cross-Site Scripting, атака, при которой злоумышленник вводит HTML или JavaScript-код, который затем отображается в браузере других пользователей.

```
<div class="form-row">
  <div class="form-group">
    <label for="service">Услуга *</label>
    <select id="service" name="service" required>
      <option value="" disabled selected>Выберите</option>
      {% for service in services %}
      <option value="{{ service.id }}">{{ service.services_name }} -- {{ service.price }}</option>
      {% endfor %}
    </select>
  </div>
</div>
```

12: реализация безопасного ввода

9. Валидация и фильтрация входных данных

Задание: проверка и фильтрация входных данных

Ход работы:

В приложении реализована проверка и фильтрация входных данных на серверной стороне. После получения данных из формы выполняется удаление лишних пробелов с помощью метода `strip()`, проверка обязательных полей, контроль длины строк и проверка типа данных. Например, идентификатор услуги проверяется как числовое значение с помощью метода `isdigit()`, после чего преобразуется в тип `int`. Дополнительно в HTML-форме используются атрибуты `required`, обеспечивающие предварительную проверку данных в браузере.

```
@app.route("/appointment", methods=["GET", "POST"])
def appointment():
    print("appointment route called")
    print(f'request method: {request.method}')

    if request.method == "POST":
        print(f"POST data: {request.form}")
        name = request.form.get("name", "").strip() # удаляем лишние пробелы
        phone = request.form.get("phone", "").strip()
        service_id = request.form.get("service", "").strip()
        date = request.form.get("date")
        time = request.form.get("time")
        comment = request.form.get("comment", "").strip()
        master_id = request.form.get("master") or 1

        if not name:
            return "Name is required", 400

        if len(phone) < 10:
            return "phone is too short", 400

        if len(phone) > 50:
            return "phone is too long", 400

        if len(name) > 100:
            return "name is too long", 400

        if not service_id or not service_id.isdigit():
            return "please select a valid service", 400
        service_id = int(service_id)

        appointment_datetime = f"{date} {time}:00"
```

13: фильтрация входных данных (1)

```

42
43     appointment_model.create_appointment(
44         client_name=name,
45         phone=phone,
46         service_id=service_id,
47         appointment_datetime=appointment_datetime,
48         client_wish=comment,
49         master_id=int(master_id) if str(master_id).isdigit() else 1
50     )
51
52
53     return redirect(url_for("appointment"))
54
55 services = service_model.get_all_services()
56 return render_template("message.html", services=services)
57

```

14: фильтрация входных данных (2)

10. Dependency Injection (DI)

Задание: Dependency Injection. Использовать DI для подключения к БД, логгера, сервисов и т.д.

Ход работы:

В проекте используется Dependency Injection для передачи подключения к базе данных в модели. Объект подключения connection создаётся в модуле app.py и передаётся в конструктор класса AppointmentModel при его создании. Модель получает готовую зависимость извне и не создаёт её самостоятельно. Такой подход снижает связность компонентов и упрощает тестирование и сопровождение приложения.

```

1 # main flask file
2 from flask import Flask, render_template
3 import requests
4
5 from database.db import connect_db
6
7
8 app = Flask(__name__)
9
10 connection = connect_db()
11 # обязательно поменять connect_db, нужно использовать пул соединений
12 # иначе для многопоточности не сгодится
13
14 from controller.home_controller import *
15 from controller.about_us_controller import *
16 from controller.appointment_controller import *
17
18
19 if __name__ == "__main__":
20     app.run(
21         host="0.0.0.0",
22         port=80,
23         debug=True
24     )
25

```

15: создание connection

```

class AppointmentModel:
    # ToDo: BaseModel for these 2 funcs
    def __init__(self, db_connection_or_func):
        """
        get_connection_func - for psycopg2 connection
        """
        if callable(db_connection_or_func):
            self.get_conn = db_connection_or_func

        self.get_conn = lambda: db_connection_or_func

```

16: передача connection в модель:

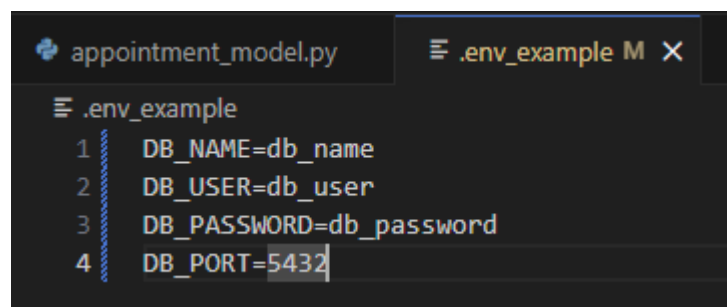
11. Конфигурация через окружение

Задание: все параметры (строки подключения к БД, ключи) хранятся в переменных окружения или конфигурационных файлах

Ход работы:

Был создан файл `.env` и сразу добавлен в `.gitignore`, чтобы не загрузить секретные данные на гитхаб.

Такой файл нужен, чтобы сохранить данные в секрете, ведь там хранятся пароли, имена, логины, порты и так далее. Для красивого гитхаба был создан `.env_example`, который уже можно пушить на GitHub, чтобы можно было узнать, какие переменные окружения вообще есть.



```

# .env_example
1 DB_NAME=db_name
2 DB_USER=db_user
3 DB_PASSWORD=db_password
4 DB_PORT=5432

```

17: .env_example

Как считываются данные, можно посмотреть на скрипте ниже. У случае, если данные из `env` не подходят, были вписаны значения по умолчанию: `localhost`, `5432`, `postgres`, `postgres`.


```
def connect_db(retries=5, delay=2):
    # обязательно поменять connect_db, нужно использовать пул соединений
    # иначе для многопоточности не сгодится
    for attempt in range(retries):
        conn = psycopg2.connect(
            host=os.getenv('DB_HOST', 'localhost'),
            port=os.getenv('DB_PORT', '5432'),
            database=os.getenv('DB_NAME', 'postgres'),
            user=os.getenv('DB_USER', 'postgres'),
            password=os.getenv('DB_PASSWORD', 'postgres')
        )
        print("database connection successful")
        return conn

    if attempt < retries - 1:
        print(f"database connection failed (attempt {attempt + 1}/{retries}). Retrying in {delay} seconds...")
        time.sleep(delay)

    print(f"error connecting to the database: {e}")
    return None
```

18: использование секретов из env